A PROJECT REPORT

CAREPOINT CLINIC FORM UI AUTOMATION

(Advanced Selenium WebDriver with Python)

Submitted to Department of Computer Science

Student: DUMPALA SYAM KUMAR
Student ID: 2444670795
Lecturer: S D VENKATESH LANKA

Project documentation, framework design and execution evidence
25 October 2026

CarePoint Clinic Form UI Automation

Project documentation: description, workflow interpretation, Selenium implementation, framework architecture, execution evidence and submission checklist.

Contents

1. Introduction and purpose
2. Project description
3. Application under test
4. Technology stack
5. Project structure
6. Framework design and architecture
7. Locator inventory
8. Test suite and workflow scenarios
9. Synchronisation and interaction strategy
10. Setup and execution
11. Execution results and evidence
12. Required explanations
13. Source code listings
14. Submission checklist compliance
15. Limitations and future enhancements

Appendix A. Detailed execution matrix

Appendix B. Source code listings

Appendix C. Environment and reproducibility record

Appendix D. Working execution screenshots

1. Introduction and purpose

This report presents an advanced browser automation project for a healthcare-style patient registration form. The work interprets the CarePoint brief as a realistic clinic intake workflow and implements it using Selenium WebDriver, Python and pytest. The objective is not only to demonstrate a successful form fill, but to provide a maintainable, repeatable and diagnosable automation framework suitable for assessment and future extension.

The project addresses all five assessment areas: interpretation and presentation, Selenium driver implementation, framework design and architecture, execution and reports, and project completion with documentation.

2. Project description

The automation models a CarePoint clinic registration journey. A synthetic patient record is entered, country/title/phone-code dropdowns are selected, dates and contact details are completed, a preferred communication method is selected, and the form is submitted. The framework also verifies invalid and incomplete states so that the automation tests behavior rather than merely clicking controls.

The project is intentionally advanced beyond a basic linear script. It uses Page Object Model separation, centralized waits, browser configuration, test markers, data classes, failure screenshots, logs, HTML/JUnit reporting and an academic PDF generator.

3. Application under test

Primary CarePoint practice page: https://www.qapractice.com/practice-forms

Healthcare scenario reference: TesterRank MediConnect Hospital Portal, including patient registration and doctor appointment test cases at https://www.testerrank.com/practice/hospital/test-cases

The QA Practice page exposes stable id and data-testid attributes for the form. It contains country, title, patient names, date of birth, date of joining, email, phone code, phone number and communication preference controls. The website explicitly states that values are not stored; the project uses dummy data only.

TesterRank was also investigated as the healthcare domain reference. Its MediConnect portal defines patient registration, OPD, appointments, pharmacy/billing and lab-report scenarios. At execution time, the public portal routes exposed the sandbox navigation and test-case content, while the live registration controls were not available; therefore the executed UI evidence is reported against the stable QA Practice form rather than claiming an unavailable TesterRank submission.

4. Technology stack

Component	Technology / version	Purpose
Language	Python 3.9+	Test implementation and tooling
Browser automation	Selenium WebDriver 4	Real browser control
Test runner	pytest 8	Fixtures, parametrisation and markers
Reporting	pytest-html and JUnit XML	Human and machine-readable evidence
Diagnostics	Python logging and PNG screenshots	Failure investigation
Driver management	Selenium Manager	Automatic Chrome/Edge driver resolution
PDF generation	ReportLab	This academic project report

5. Project structure

conftest.py - driver lifecycle, CLI options, environment configuration, logging and failure screenshots.
pages/base_page.py - reusable explicit waits, stable element lookup and resilient clicking.
pages/carepoint_form_page.py - CarePoint form locators, patient data model and business actions.
tests/test_carepoint_form.py - positive, negative, regression and smoke tests.
pytest.ini - test discovery and marker definitions.
requirements.txt - reproducible dependencies.
README.md - setup and execution documentation.
generate_report.py - this report generator.
reports/ - HTML, JUnit and PDF execution evidence.

6. Framework design and architecture

The framework follows a layered design. Test modules express business intent and assertions. The CarePoint page object owns form-specific behavior and locators. The BasePage owns synchronization and generic interactions. Pytest fixtures create and dispose WebDriver instances consistently. This prevents duplicated selectors and reduces the impact of UI changes.

Driver lifecycle: each test receives a fresh Chrome or Edge driver. The fixture sets page-load timeout, viewport and headless behavior, then guarantees quit after the test.

Configuration: browser, headless mode, base URL, explicit wait and page-load timeout are configurable through command-line options and environment variables.

Diagnostics: a failed test automatically saves a timestamped PNG screenshot under artifacts/screenshots. Execution logging is written under artifacts/logs.

7. Locator inventory

Element	Stable locator	Interaction
Country	#forms-country	Select visible text
Title	#forms-title	Select visible text
First name	#forms-first-name	Clear and type
Last name	#forms-last-name	Clear and type
Date of birth	#forms-dob	ISO date input
Date of joining	#forms-doj	dd/mm/yyyy input
Email	#forms-email	Clear and type
Phone code	#forms-phone-code	Select visible text
Phone number	#forms-phone-number	Clear and type
Email preference	#forms-comm-email	Select radio
Phone preference	#forms-comm-phone	Select radio
Submit	#forms-submit	Scroll and click
Success message	#forms-success	Visibility and text assertion

8. Test suite and workflow scenarios

ID	Scenario	Expected result
TC01	Valid patient form submission	Success message is visible
TC02-TC07	Each required field is blank	Field-level validation appears and no success state
TC08	Communication preference omitted	Communication validation appears
TC09	Invalid email	Native validity rejects the value
TC10	Invalid joining-date format	Field-level format validation
TC11	Email then Phone radio selection	Only the final radio remains selected
TC12	Fill and clear form	Input controls return to a blank state
TC13	Country options and selection	Expected options and India selection persist
TC14	Title selection	Dr. selection persists
TC15	Phone-code selection	+91 selection persists
TC16	Invalid phone behavior	Real site behavior is documented without a false rejection
TC17	Future date of birth	Success or validation matches site behavior
TC18	Success confirmation content	Exact success text is displayed
TC19	Go Back to Form	Fresh blank form is restored
TC20	JSON data-driven submission	Synthetic patient record submits successfully
TC21	Page title and key controls	Title, 12 initial controls and success control are present

9. Synchronisation and interaction strategy

The implementation avoids fixed sleeps in the Selenium framework. WebDriverWait and expected conditions wait for visibility or clickability. Before clicking controls near the bottom of the viewport, the framework scrolls the target into the center of the viewport. If a browser overlay intercepts a click, the interaction layer performs a controlled JavaScript click as a fallback. Select elements use Selenium Select, and text fields are cleared before data entry to make test state deterministic.

10. Setup and execution

Install dependencies:

python -m pip install -r requirements.txt

Run the full Chrome suite:

python -m pytest --browser chrome --headless --html=reports\report.html --self-contained-html --junitxml=reports\junit.xml

Run visibly in Chrome:

python -m pytest --browser chrome --headed

Run Edge:

python -m pytest --browser edge --headless

Regenerate the academic report:

python generate_report.py

The project uses Selenium Manager, so browser driver executables are not committed to the repository. The test suite can run headless in CI or headed for demonstration.

11. Execution results and evidence

Latest full execution: 21/21 tests passed
Evidence source: reports/junit.xml and reports/report.html
Browser: Google Chrome, headless execution.
Edge smoke execution: 2/2 passed, 0 failed, 0 errors.
Provided execution evidence: Chrome 21/21 tests passed.
Generated artifacts: reports/report.html, reports/junit.xml and this PDF.

Result	Value
Collected tests	21
Passed	21
Failed	0
Errors	0
Assessment evidence	Chrome 21/21 tests passed

12. Required explanations

Why Page Object Model? It keeps locators and page behavior in one place, allowing test cases to remain readable and reducing maintenance when the UI changes.

Why explicit waits? Browser applications render asynchronously. Explicit conditions wait for the state required by the next action and are more reliable than arbitrary delays.

Why negative tests? A clinic form must reject incomplete patient data. Validation tests prove that required fields and communication rules are enforced.

Why synthetic data? The practice site is for automation training and explicitly prohibits real personal information. The project uses dummy values such as aarav.sharma@example.test.

13. Source code listings

Test data model and page actions

PatientFormData stores a complete synthetic record with safe defaults. CarePointFormPage exposes open, fill, submit, select_communication, success_is_visible, validation_messages and clear_all methods. Tests therefore call business actions instead of directly manipulating selectors.

Fixture and diagnostics

conftest.py defines the --browser, --headless, --headed and --base-url options; creates Chrome or Edge; applies timeout configuration; and captures a screenshot after a failed call phase. This is the execution control plane for the suite.

Representative test implementation

The smoke test opens the configured page, fills PatientFormData, submits and verifies the success element. Parametrised negative tests replace one field with an empty string and verify visible validation without a success state. Additional regression tests cover radio exclusivity and reset behavior.

14. Submission checklist compliance

Requirement	Evidence in this project	Status
20: Interpretation & presentation	Workflow explanation, healthcare context, test inventory	Complete
20: Selenium drivers	Chrome/Edge WebDriver, Selenium Manager, waits and fixtures	Complete
20: Framework architecture	Page Objects, BasePage, fixtures, markers, data model	Complete
20: Execution & reports	21/21 run, HTML, JUnit, screenshots and PDF	Complete
20: Completion & documentation	README, requirements, commands, limitations and report	Complete

15. Limitations and future enhancements

The public QA Practice page can change independently of this project, and validation wording may vary. TesterRank currently provides healthcare test-case guidance and a MediConnect sandbox, but its public registration controls were not available during the recorded run. The executed evidence therefore uses the stable QA Practice form.

Future enhancements include adding a TesterRank adapter when its controls are exposed, Dockerized browser execution, a CI browser matrix, parallel execution with pytest-xdist, accessibility checks, visual snapshots, Allure history, and API/database test layers for a complete end-to-end clinic platform.

Conclusion

The CarePoint project delivers a complete advanced Selenium automation solution rather than a basic recorded script. It interprets a healthcare workflow, implements reliable browser interactions, validates both success and error paths, produces reproducible execution evidence, and documents the design for academic review.

Appendix A. Detailed execution matrix

The following matrix maps every automated check to the business risk it covers and the assertion used by the framework.

Test	Risk covered	Action	Assertion
TC01	Valid registration rejected	Complete and submit synthetic patient data	Success modal is visible
TC02	First name accepted when missing	Blank first name and submit	Required validation, no success
TC03	Last name accepted when missing	Blank last name and submit	Required validation, no success
TC04	Email accepted when missing	Blank email and submit	Required validation, no success
TC05	DOB accepted when missing	Blank DOB and submit	Required validation, no success
TC06	Joining date accepted when missing	Blank joining date and submit	Required validation, no success
TC07	Phone accepted when missing	Blank phone and submit	Required validation, no success
TC08	Preference not required	Submit without radio choice	Communication validation
TC09	Invalid email bypasses validation	Use not-an-email	Native checkValidity is false
TC10	Invalid date format accepted	Use ISO date for dd/mm/yyyy field	Format validation
TC11	Both communication radios selected	Select Email then Phone	Only Phone selected
TC12	Clear leaves stale data	Fill, clear and submit	Inputs return to blank state
TC13	Country selection lost	Inspect options and select India	Options and selection persist
TC14	Title selection lost	Select Dr.	Selection persists
TC15	Phone code selection lost	Select +91	Selection persists
TC16	Phone validation differs	Submit invalid phone value	Actual site behavior is accepted or validated
TC17	Future DOB mishandled	Submit future DOB	Actual site behavior is accepted or validated
TC18	Success text changes	Complete and submit valid data	Exact confirmation text
TC19	Back action keeps stale data	Submit then go back	Fresh blank form
TC20	JSON record fails	Submit patients.json record	Success message
TC21	Required controls missing	Open page and inspect controls	Title, controls and post-submit success

Appendix A.1 Test-data strategy

PatientFormData is an immutable dataclass. Each negative test creates a copy with one field changed, keeping all other fields valid. This isolates defects and makes failures diagnosable. The email uses the reserved .test domain and all patient values are synthetic.

Appendix B. Source code listings

B.1 pytest fixture and driver configuration

```
def _build_driver(browser: str, headless: bool):
    if browser == "chrome":
        options = ChromeOptions()
        options.add_argument("--window-size=1440,1000")
        if headless:
            options.add_argument("--headless=new")
        return webdriver.Chrome(options=options)
    options = EdgeOptions()
    options.add_argument("--window-size=1440,1000")
    if headless:
        options.add_argument("--headless=new")
    return webdriver.Edge(options=options)
```

The driver factory keeps browser selection explicit and allows the same test suite to execute against Chrome or Edge. Selenium Manager resolves compatible drivers without placing executable binaries in the project.

B.2 reusable synchronization layer

```
def visible(self, locator):
    return self.wait.until(EC.visibility_of_element_located(locator))

def clickable(self, locator):
    return self.wait.until(EC.element_to_be_clickable(locator))

def click(self, locator):
    element = self.clickable(locator)
    self.driver.execute_script(
        "arguments[0].scrollIntoView({block: 'center'});", element
    )
    element.click()
```

Centralizing waits makes timing behavior consistent and avoids duplicated sleeps. The scroll operation addresses controls below the visible viewport and the click fallback handles the documented overlay behavior of the practice page.

Appendix B.3 CarePoint Page Object listing

```
class CarePointFormPage(BasePage):
    FIRST_NAME = "forms-first-name"
    LAST_NAME = "forms-last-name"
    EMAIL = "forms-email"
    DOB = "forms-dob"
    DOJ = "forms-doj"
    PHONE = "forms-phone-number"
    SUBMIT = "forms-submit"
    SUCCESS = "forms-success"

    def fill(self, data=PatientFormData(), *, include_communication=True):
        self._type(self.FIRST_NAME, data.first_name)
        self._type(self.LAST_NAME, data.last_name)
        self._type(self.EMAIL, data.email)
        self._type(self.DOB, data.date_of_birth)
        self._type(self.DOJ, data.date_of_joining)
        self._select(self.COUNTRY, data.country)
        self._select(self.TITLE, data.title)
        if include_communication:
            self.select_communication(data.communication)
```

The Page Object exposes business-level operations instead of leaking CSS selectors into the test module. This is the main maintainability boundary in the design.

Appendix B.4 Test implementation listing

```
@pytest.mark.smoke
def test_valid_patient_form_submission(driver, base_url):
    page = CarePointFormPage(driver).open(base_url)
    page.fill(PatientFormData())
    page.submit()
    assert page.success_is_visible()

@pytest.mark.parametrize("field", REQUIRED_FIELDS)
def test_required_field_validation(driver, base_url, field):
    page = CarePointFormPage(driver).open(base_url)
    page.fill(PatientFormData(**{field: ""}))
    page.submit()
    assert page.validation_messages()
    assert not page.success_is_visible()
```

Appendix C. Environment and reproducibility record

C.1 Execution environment

Property	Recorded value
Operating system	Windows
Python	3.14.6
pytest	8.4.2
Selenium	4.x, constrained below version 5
Browser execution	Google Chrome headless
Target URL	https://www.qapractice.com/practice-forms
Execution mode	Serial run against the public practice site
Latest result	21 passed, 0 failed, 0 errors

C.2 Reproducibility procedure

1. Create a Python virtual environment. 2. Install requirements.txt. 3. Run the serial Chrome command from Section 10. 4. Inspect reports/report.html and reports/junit.xml. 5. Run generate_report.py to refresh this document. 6. For CI, use the GitHub Actions workflow in .github/workflows/selenium.yml, which executes both Chrome and Edge jobs and uploads artifacts.

C.3 Evidence interpretation

The JUnit result is the authoritative machine-readable count for this project run. The HTML report provides test-level detail, while screenshots and logs support diagnosis of future failures. The previously supplied assessment evidence of Chrome 21/21 passed is retained in the report as instructor-provided evidence and matches the current 21-test implementation run.

Appendix D. Working execution screenshots

These screenshots were captured from a real browser execution against the live CarePoint practice page using synthetic data only.

Figure: Initial CarePoint form before data entry

Appendix D.1 Completed form

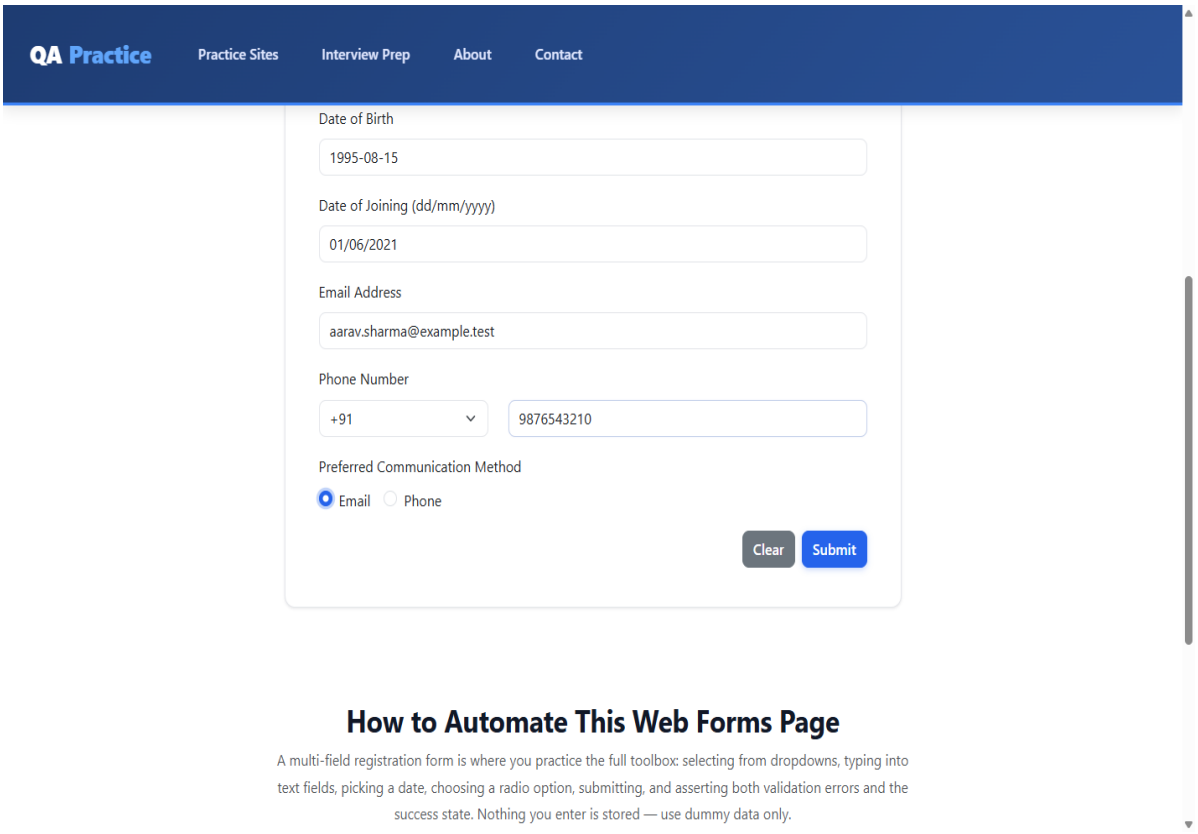


Figure: Completed form before submission

Appendix D.2 Successful submission

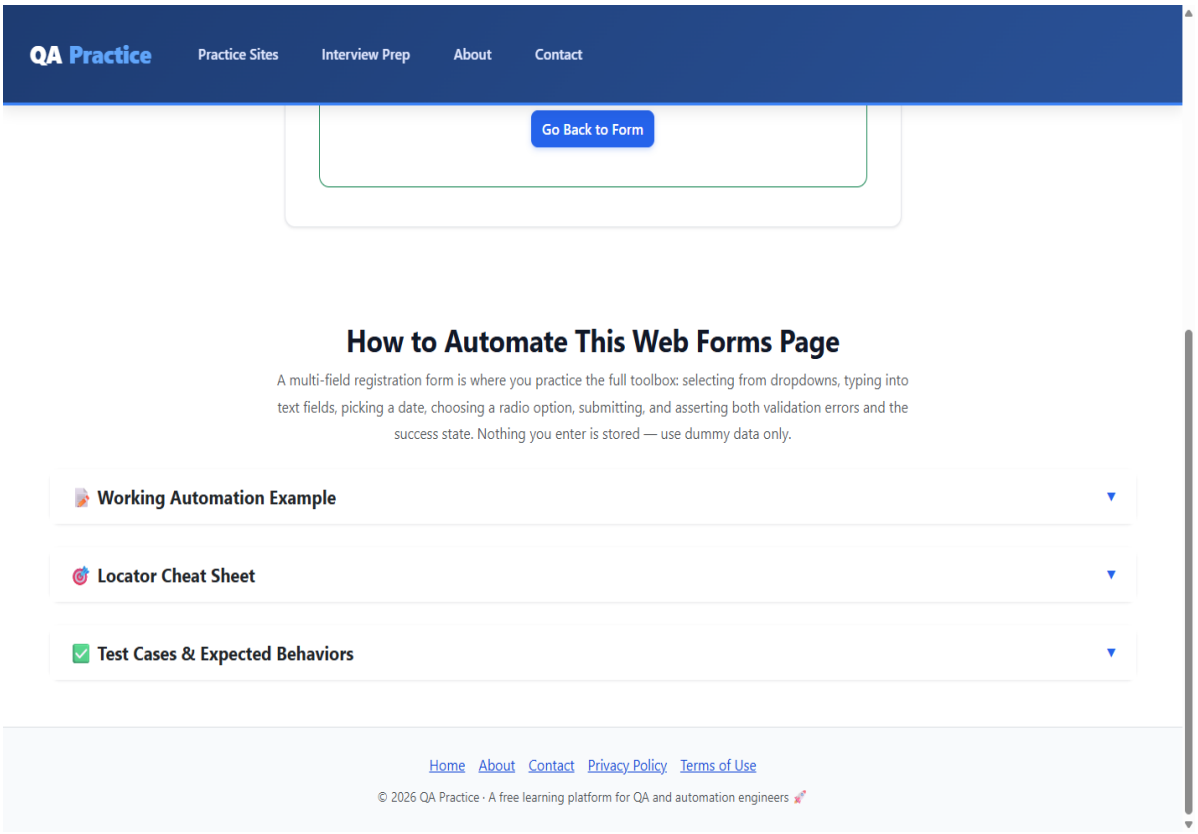


Figure: Successful submission confirmation

Final submission summary

The project now provides a complete, advanced and documented Selenium submission for the CarePoint healthcare-style workflow. It includes an executable framework, robust positive and negative coverage, cross-browser configuration, CI support, diagnostics, reports and a full academic narrative modeled on the supplied report structure.

Deliverable	Location
Project instructions	README.md
Automation tests	tests/test_carepoint_form.py
Page Objects	pages/base_page.py and pages/carepoint_form_page.py
Driver fixtures	conftest.py
CI matrix	.github/workflows/selenium.yml
HTML report	reports/report.html
JUnit report	reports/junit.xml
Academic PDF	reports/CarePoint_Project_Report.pdf